

IFRN — Campus Natal Central

Tecnologia em Análise e Desenvolvimento de Sistemas

Programação e Administração de Banco de Dados

APIs REST com Node.js e Express

Do conceito básico à integração com PostgreSQL

Sumário

1. Fundamentos de API e HTTP
2. O que é Node.js?
3. Instalando o Express
4. Primeiros Endpoints — GET e POST sem estado
5. Organizando Rotas
6. Middlewares
7. Models — Organizando a Lógica de Dados
8. Conexão com PostgreSQL via Sequelize
9. Migrações de Banco de Dados
10. API Completa: Exemplo Integrado
11. Atividade Prática

1. Fundamentos de API e HTTP

1.1 O que é uma API?

API significa Application Programming Interface — uma interface que permite que diferentes sistemas se comuniquem entre si. No contexto web, uma API REST (Representational State Transfer) é um conjunto de regras que define como um cliente (como um aplicativo ou navegador) pode solicitar dados ou executar ações em um servidor.



Conceito Central

Uma API REST funciona como um garçom em um restaurante: o cliente (você) faz pedidos, o garçom (API) leva o pedido à cozinha (servidor/banco de dados) e traz de volta o resultado.

O protocolo HTTP é o idioma usado nessa comunicação.

1.2 Verbos HTTP

Cada ação em uma API REST é representada por um verbo HTTP:

Verbo	Ação CRUD	Descrição
GET	Read (Ler)	Buscar dados do servidor
POST	Create (Criar)	Enviar novos dados ao servidor
PUT / PATCH	Update (Atualizar)	Alterar dados existentes
DELETE	Delete (Excluir)	Remover dados do servidor

1.3 Códigos de Status HTTP

O servidor sempre retorna um código numérico indicando o resultado da requisição:

Código	Família	Exemplos comuns
2xx	Sucesso	200 OK · 201 Created · 204 No Content
3xx	Redirecionamento	301 Moved · 304 Not Modified
4xx	Erro do cliente	400 Bad Request · 401 Unauthorized · 404 Not Found
5xx	Erro do servidor	500 Internal Server Error · 503 Service Unavailable

2. O que é Node.js?

Node.js é um ambiente de execução JavaScript no lado do servidor. Isso significa que podemos usar a mesma linguagem (JavaScript) tanto no frontend (navegador) quanto no backend (servidor). O Node.js é baseado no motor V8 do Google Chrome e é conhecido por sua alta performance e modelo assíncrono (non-blocking I/O).

💡 Por que Node.js para APIs?

- ✓ JavaScript: linguagem que muitos já conhecem do frontend
- ✓ Performance: ideal para operações de entrada/saída (I/O) como acesso a banco de dados
- ✓ NPM: maior ecossistema de pacotes do mundo
- ✓ Assíncrono: trata múltiplas requisições simultâneas com eficiência

2.1 Verificando a instalação

Antes de começar, verifique se o Node.js está instalado no seu sistema:

Terminal

```
# Verificar a versão do Node.js
node --version

# Verificar a versão do NPM (Node Package Manager)
npm --version

# Resultado esperado (versões podem variar):
# v20.x.x
# 10.x.x
```

2.2 Iniciando um projeto Node.js

Todo projeto Node.js começa com a criação do arquivo package.json, que gerencia as dependências e configurações do projeto:

Terminal

```
# Criar pasta do projeto
mkdir minha-api
cd minha-api

# Inicializar o projeto (gera o package.json)
npm init -y
```

3. Instalando o Express

Express é o framework web mais popular para Node.js. Ele simplifica a criação de servidores HTTP e de APIs REST, fornecendo um conjunto de ferramentas para gerenciar rotas, middlewares e requisições/respostas.

3.1 Instalação

Terminal

```
# Instalar o Express como dependência do projeto
npm install express

# O package.json será atualizado automaticamente:
# {
#   "dependencies": {
#     "express": "^4.18.2"
#   }
# }
```

3.2 Servidor mínimo com Express

Vamos criar o arquivo mais simples possível — um servidor que responde 'Olá, Mundo!' a qualquer requisição:

index.js

```
// index.js
const express = require('express');
const app = express();

// Define a porta onde o servidor vai escutar
const PORT = 3000;

// Rota raiz: GET /
app.get('/', (req, res) => {
  res.send('Olá, Mundo! Minha primeira API com Express.');
```

```
});

// Inicia o servidor
app.listen(PORT, () => {
  console.log(`Servidor rodando em http://localhost:${PORT}`);
});
```

Terminal

```
# Executar o servidor
node index.js

# Saída esperada:
# Servidor rodando em http://localhost:3000
```

Como testar?

1. Abra o navegador e acesse: `http://localhost:3000`
2. Ou use o curl no terminal: `curl http://localhost:3000`
3. Você verá a mensagem: Olá, Mundo! Minha primeira API com Express.

Parabéns! Você acabou de criar seu primeiro servidor com Node.js e Express.

4. Primeiros Endpoints — GET e POST sem estado

Vamos agora criar uma API que simula um catálogo de produtos. Inicialmente, os dados ficarão na memória (sem banco de dados) — isso é o que chamamos de API sem estado persistente (stateless storage). É a forma mais simples de aprender os conceitos antes de conectar um banco.

4.1 Habilitando JSON no Express

Para que o Express consiga ler dados no formato JSON enviados pelo cliente, precisamos registrar o middleware `json()`:

index.js

```
// index.js — versão atualizada
const express = require('express');
const app = express();
const PORT = 3000;

// Middleware: habilita leitura de JSON no corpo das requisições
app.use(express.json());

app.listen(PORT, () => {
  console.log(`Servidor rodando em http://localhost:${PORT}`);
});
```

4.2 Endpoint GET — Listar todos os produtos

index.js — continuação

```
// Dados em memória (array simula o banco de dados por enquanto)
let produtos = [
  { id: 1, nome: 'Notebook', preco: 2500.00 },
  { id: 2, nome: 'Mouse', preco: 89.90 },
  { id: 3, nome: 'Teclado', preco: 149.00 },
];

// GET /produtos — retorna todos os produtos
app.get('/produtos', (req, res) => {
  res.status(200).json(produtos);
});
```

4.3 Endpoint GET — Buscar produto por ID

Usamos `:id` como parâmetro de rota (route parameter) — o Express extrai automaticamente o valor da URL:

index.js — continuação

```
// GET /produtos/:id — retorna um produto específico
app.get('/produtos/:id', (req, res) => {
```

```

const id = parseInt(req.params.id); // converte string para número
const produto = produtos.find(p => p.id === id);

if (!produto) {
  return res.status(404).json({ mensagem: 'Produto não encontrado' });
}

res.status(200).json(produto);
});

```

4.4 Endpoint POST — Criar novo produto

No POST, os dados vêm no corpo (body) da requisição. O middleware `express.json()` que configuramos anteriormente torna esses dados disponíveis em `req.body`:

index.js — continuação

```

// POST /produtos - cria um novo produto
app.post('/produtos', (req, res) => {
  const { nome, preco } = req.body;

  // Validação básica
  if (!nome || preco === undefined) {
    return res.status(400).json({ mensagem: 'Nome e preço são obrigatórios' });
  }

  // Gera um ID incremental simples
  const novoId = produtos.length > 0
    ? Math.max(...produtos.map(p => p.id)) + 1
    : 1;

  const novoProduto = { id: novoId, nome, preco };
  produtos.push(novoProduto);

  res.status(201).json(novoProduto); // 201 Created
});

```

Terminal — Testando com curl

```

# Testar GET - listar todos
curl http://localhost:3000/produtos

# Testar GET - buscar por ID
curl http://localhost:3000/produtos/1

# Testar POST - criar produto
curl -X POST http://localhost:3000/produtos \
  -H "Content-Type: application/json" \
  -d '{"nome": "Monitor", "preco": 899.00}'

```


5. Organizando Rotas

Quando a API cresce, manter tudo em um único arquivo se torna difícil de manter. O Express nos permite separar as rotas em arquivos dedicados usando o Router:

5.1 Estrutura de pastas recomendada

Estrutura de Diretórios

```
minha-api/  
├─ index.js           # Entrada principal  
├─ package.json  
└─ src/  
    └─ routes/  
        └─ produtos.js # Rotas de produtos
```

5.2 Arquivo de rotas

src/routes/produtos.js

```
// src/routes/produtos.js  
  
const express = require('express');  
const router = express.Router(); // cria um roteador isolado  
  
let produtos = [  
  { id: 1, nome: 'Notebook', preco: 2500.00 },  
  { id: 2, nome: 'Mouse',      preco: 89.90 },  
];  
  
// GET /produtos  
router.get('/', (req, res) => {  
  res.json(produtos);  
});  
  
// GET /produtos/:id  
router.get('/:id', (req, res) => {  
  const produto = produtos.find(p => p.id === parseInt(req.params.id));  
  if (!produto) return res.status(404).json({ mensagem: 'Não encontrado' });  
  res.json(produto);  
});  
  
// POST /produtos  
router.post('/', (req, res) => {  
  const { nome, preco } = req.body;  
  if (!nome || preco === undefined)  
    return res.status(400).json({ mensagem: 'Dados inválidos' });  
  const novo = { id: Date.now(), nome, preco };  
  produtos.push(novo);  
  res.status(201).json(novo);  
});
```

```
module.exports = router;
```

5.3 Registrando as rotas no index.js

index.js

```
// index.js - versão com rotas separadas
const express = require('express');
const app = express();
const PORT = 3000;

const produtosRouter = require('./src/routes/produtos');

app.use(express.json());

// Registra o roteador com prefixo /produtos
app.use('/produtos', produtosRouter);

app.listen(PORT, () => {
  console.log(`Servidor rodando em http://localhost:${PORT}`);
});
```

6. Middlewares

Middleware é uma função que fica no meio do caminho entre a requisição do cliente e a resposta do servidor. É um dos conceitos mais importantes do Express, pois permite interceptar, modificar ou validar requisições de forma modular e reutilizável.

Como funciona o fluxo?

Requisição → [Middleware 1] → [Middleware 2] → [Rota] → Resposta

Cada middleware recebe três parâmetros:

- req — objeto da requisição (Request)
- res — objeto da resposta (Response)
- next — função que passa para o próximo middleware

6.1 Middleware de Log

Um middleware de log registra informações sobre cada requisição recebida — muito útil para depuração:

src/middlewares/logger.js

```
// src/middlewares/logger.js
function logger(req, res, next) {
  const agora = new Date().toISOString();
  console.log(`[${agora}] ${req.method} ${req.url}`);
  next(); // OBRIGATÓRIO: passa para o próximo middleware/rota
}

module.exports = logger;
```

6.2 Middleware de Validação

Podemos criar middlewares para validar dados antes de chegarem à rota, mantendo as rotas mais limpas:

src/middlewares/validarProduto.js

```
// src/middlewares/validarProduto.js
function validarProduto(req, res, next) {
  const { nome, preco } = req.body;

  if (!nome || typeof nome !== 'string' || nome.trim() === '') {
    return res.status(400).json({ erro: 'O campo "nome" é obrigatório' });
  }

  if (preco === undefined || typeof preco !== 'number' || preco < 0) {
```

```

    return res.status(400).json({ erro: 'O campo "preco" deve ser número positivo' });
  }

  next(); // dados válidos, segue para a rota
}

module.exports = validarProduto;

```

6.3 Registrando middlewares

Os middlewares podem ser globais (aplicados a todas as rotas) ou locais (aplicados apenas a uma rota específica):

index.js

```

// index.js - usando middlewares
const express = require('express');
const app = express();
const PORT = 3000;

const logger      = require('./src/middlewares/logger');
const validarProduto = require('./src/middlewares/validarProduto');
const produtosRouter = require('./src/routes/produtos');

app.use(express.json());

// Middleware GLOBAL - executa em TODAS as requisições
app.use(logger);

app.use('/produtos', produtosRouter);

app.listen(PORT, () => {
  console.log(`Servidor rodando em http://localhost:${PORT}`);
});

```

src/routes/produtos.js — middleware local

```

// src/routes/produtos.js - usando middleware local no POST
const validarProduto = require('../middlewares/validarProduto');

// Middleware LOCAL - executa SOMENTE nesta rota POST
router.post('/', validarProduto, (req, res) => {
  const { nome, preco } = req.body;
  const novo = { id: Date.now(), nome, preco };
  produtos.push(novo);
  res.status(201).json(novo);
});

```

7. Models — Organizando a Lógica de Dados

À medida que a API cresce, misturar a lógica de dados diretamente nas rotas torna o código difícil de manter e testar. O padrão MVC (Model-View-Controller) sugere separar as responsabilidades em camadas.

Camada	Responsabilidade	No Express
Model	Acesso e manipulação dos dados	Arquivos em <code>src/models/</code>
Controller	Lógica de negócio e orquestração	Arquivos em <code>src/controllers/</code>
Route	Mapeamento de URLs para controllers	Arquivos em <code>src/routes/</code>

7.1 Estrutura de pastas com Model e Controller

Estrutura de Diretórios

```
minha-api/
├─ index.js
├─ package.json
└─ src/
   └─ models/
      └─ ProdutoModel.js      # Acesso a dados
   └─ controllers/
      └─ ProdutoController.js # Lógica de negócio
   └─ middlewares/
      └─ logger.js
      └─ validarProduto.js
   └─ routes/
      └─ produtos.js          # Apenas mapeia URLs
```

7.2 Model

O Model encapsula o acesso aos dados. Por ora, ainda usamos array em memória, mas a lógica de acesso fica centralizada — depois, só trocaremos a implementação do Model pelo banco de dados:

`src/models/ProdutoModel.js`

```
// src/models/ProdutoModel.js
let produtos = [
  { id: 1, nome: 'Notebook', preco: 2500.00 },
  { id: 2, nome: 'Mouse',    preco: 89.90 },
];

const ProdutoModel = {
  findAll()      { return produtos; },

  findById(id)   { return produtos.find(p => p.id === id); },
}
```

```

create(dados) {
  const novo = { id: Date.now(), ...dados };
  produtos.push(novo);
  return novo;
},

update(id, dados) {
  const idx = produtos.findIndex(p => p.id === id);
  if (idx === -1) return null;
  produtos[idx] = { ...produtos[idx], ...dados };
  return produtos[idx];
},

destroy(id) {
  const idx = produtos.findIndex(p => p.id === id);
  if (idx === -1) return false;
  produtos.splice(idx, 1);
  return true;
},
};

module.exports = ProdutoModel;

```

7.3 Controller

 **src/controllers/ProdutoController.js**

```

// src/controllers/ProdutoController.js
const ProdutoModel = require('../models/ProdutoModel');

const ProdutoController = {
  listar(req, res) {
    const produtos = ProdutoModel.findAll();
    res.json(produtos);
  },

  buscar(req, res) {
    const id = parseInt(req.params.id);
    const produto = ProdutoModel.findById(id);
    if (!produto) return res.status(404).json({ erro: 'Não encontrado' });
    res.json(produto);
  },

  criar(req, res) {
    const novo = ProdutoModel.create(req.body);
    res.status(201).json(novo);
  },

  atualizar(req, res) {
    const id = parseInt(req.params.id);
    const atualizado = ProdutoModel.update(id, req.body);
  }
};

```

```

        if (!atualizado) return res.status(404).json({ erro: 'Não encontrado' });
        res.json(atualizado);
    },

    remover(req, res) {
        const id = parseInt(req.params.id);
        const ok = ProdutoModel.destroy(id);
        if (!ok) return res.status(404).json({ erro: 'Não encontrado' });
        res.status(204).send(); // 204 No Content
    },
};

module.exports = ProdutoController;

```

7.4 Rota simplificada com Controller

src/routes/produtos.js

```

// src/routes/produtos.js - versão com controller
const express = require('express');
const router = express.Router();
const ProdutoController = require('../controllers/ProdutoController');
const validarProduto = require('../middlewares/validarProduto');

router.get('/', ProdutoController.listar);
router.get('/:id', ProdutoController.buscar);
router.post('/', validarProduto, ProdutoController.criar);
router.put('/:id', validarProduto, ProdutoController.atualizar);
router.delete('/:id', ProdutoController.remover);

module.exports = router;

```

8. Conexão com PostgreSQL via Sequelize

Sequelize é um ORM (Object-Relational Mapper) para Node.js que suporta PostgreSQL, MySQL, SQLite e outros bancos. Ele nos permite trabalhar com o banco de dados usando objetos JavaScript, sem precisar escrever SQL puro na maioria das situações.

O que é um ORM?

ORM mapeia tabelas do banco para classes/objetos JavaScript.

Em vez de escrever: `SELECT * FROM produtos WHERE id = 1`

Você escreve: `Produto.findPk(1)`

Benefícios: código mais legível, segurança contra SQL Injection, portabilidade entre diferentes bancos de dados.

8.1 Instalação

Terminal

```
# Instalar Sequelize e o driver do PostgreSQL
npm install sequelize pg pg-hstore

# Instalar a CLI do Sequelize (ferramenta de linha de comando)
npm install --save-dev sequelize-cli
```

8.2 Configurando a conexão

Vamos organizar a configuração do banco em um arquivo separado:

src/database/connection.js

```
// src/database/connection.js
const { Sequelize } = require('sequelize');

const sequelize = new Sequelize({
  dialect: 'postgres',
  host: process.env.DB_HOST || 'localhost',
  port: process.env.DB_PORT || 5432,
  database: process.env.DB_NAME || 'pabd_db',
  username: process.env.DB_USER || 'postgres',
  password: process.env.DB_PASSWORD || 'senha123',
  logging: false, // desabilita logs SQL (mude para console.log para ver as queries)
});

module.exports = sequelize;
```


8.3 Testando a conexão

index.js

```
// index.js - testando a conexão com o banco

const express = require('express');
const sequelize = require('../src/database/connection');
const app = express();
const PORT = 3000;

app.use(express.json());

// Testar conexão ao iniciar o servidor
async function iniciar() {
  try {
    await sequelize.authenticate();
    console.log('✓ Conectado ao PostgreSQL com sucesso!');

    const produtosRouter = require('../src/routes/produtos');
    app.use('/produtos', produtosRouter);

    app.listen(PORT, () => {
      console.log(`Servidor rodando em http://localhost:${PORT}`);
    });
  } catch (erro) {
    console.error('✗ Erro ao conectar ao banco:', erro.message);
    process.exit(1);
  }
}

iniciar();
```

9. Migrações de Banco de Dados

Migração (migration) é um controle de versão para o banco de dados. Em vez de criar as tabelas manualmente, escrevemos arquivos de migração que descrevem as alterações — e o Sequelize aplica ou reverte essas alterações de forma controlada.

✓ Por que usar migrações?

- Rastreabilidade: cada alteração no banco é registrada e versionada
- Colaboração: a equipe aplica as mesmas alterações sem conflitos
- Segurança: é possível desfazer uma migração (rollback)
- Automação: pode-se aplicar todas as migrações ao subir o servidor

9.1 Configurando o Sequelize CLI

.sequelizerc

```
// .sequelizerc - configura as pastas do projeto
const path = require('path');

module.exports = {
  'config':      path.resolve('src/database/config.json'),
  'models-path': path.resolve('src/models'),
  'migrations-path': path.resolve('src/database/migrations'),
  'seeders-path': path.resolve('src/database/seeders'),
};
```

src/database/config.json

```
// src/database/config.json
{
  "development": {
    "username": "postgres",
    "password": "senha123",
    "database": "pabd_db",
    "host":     "127.0.0.1",
    "dialect":  "postgres"
  }
}
```

9.2 Criando uma migração

Terminal

```
# Inicializar estrutura do Sequelize
npx sequelize-cli init

# Criar uma migração para a tabela de produtos
```

```
npx sequelize-cli migration:generate --name criar-tabela-produtos
```

src/database/migrations/YYYYMMDDHHMMSS-criar-tabela-produtos.js

```
// src/database/migrations/YYYYMMDDHHMMSS-criar-tabela-produtos.js
'use strict';

module.exports = {
  // up: o que fazer ao aplicar a migração
  async up(queryInterface, Sequelize) {
    await queryInterface.createTable('produtos', {
      id: {
        allowNull: false,
        autoIncrement: true,
        primaryKey: true,
        type: Sequelize.INTEGER,
      },
      nome: {
        allowNull: false,
        type: Sequelize.STRING(120),
      },
      preco: {
        allowNull: false,
        type: Sequelize.DECIMAL(10, 2),
      },
      createdAt: {
        allowNull: false,
        type: Sequelize.DATE,
      },
      updatedAt: {
        allowNull: false,
        type: Sequelize.DATE,
      },
    });
  },

  // down: o que fazer ao reverter a migração
  async down(queryInterface) {
    await queryInterface.dropTable('produtos');
  },
};
```

Terminal — comandos de migração

```
# Aplicar todas as migrações pendentes
```

```
npx sequelize-cli db:migrate
```

```
# Reverter a última migração (rollback)
```

```
npx sequelize-cli db:migrate:undo
```

```
# Reverter TODAS as migrações
```

```
npx sequelize-cli db:migrate:undo:all
```


10. API Completa: Exemplo Integrado

Agora vamos atualizar o Model de Produto para usar o Sequelize (banco de dados real) em vez do array em memória. A beleza da arquitetura em camadas é que apenas o Model precisa mudar — as rotas e controllers permanecem intactos.

10.1 Model com Sequelize

src/models/Produto.js

```
// src/models/Produto.js - model com Sequelize
const { DataTypes } = require('sequelize');
const sequelize = require('../database/connection');

const Produto = sequelize.define('Produto', {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true,
  },
  nome: {
    type: DataTypes.STRING(120),
    allowNull: false,
    validate: {
      notEmpty: { msg: 'O nome não pode ser vazio' },
      len: { args: [2, 120], msg: 'Nome deve ter entre 2 e 120 caracteres' },
    },
  },
  preco: {
    type: DataTypes.DECIMAL(10, 2),
    allowNull: false,
    validate: {
      isDecimal: { msg: 'Preço deve ser um número decimal' },
      min: { args: [0], msg: 'Preço não pode ser negativo' },
    },
  },
}, {
  tableName: 'produtos', // nome exato da tabela no banco
  timestamps: true, // cria createdAt e updatedAt automaticamente
});

module.exports = Produto;
```

10.2 Controller com Sequelize

Atualizamos o controller para usar as operações assíncronas do Sequelize:

src/controllers/ProdutoController.js

```
// src/controllers/ProdutoController.js - versão com banco de dados
const Produto = require('../models/Produto');
```

```
const ProdutoController = {
  async listar(req, res) {
    try {
      const produtos = await Produto.findAll();
      res.json(produtos);
    } catch (erro) {
      res.status(500).json({ erro: 'Erro ao buscar produtos' });
    }
  },

  async buscar(req, res) {
    try {
      const produto = await Produto.findByPk(req.params.id);
      if (!produto) return res.status(404).json({ erro: 'Não encontrado' });
      res.json(produto);
    } catch (erro) {
      res.status(500).json({ erro: 'Erro ao buscar produto' });
    }
  },

  async criar(req, res) {
    try {
      const produto = await Produto.create(req.body);
      res.status(201).json(produto);
    } catch (erro) {
      if (erro.name === 'SequelizeValidationError') {
        const msgs = erro.errors.map(e => e.message);
        return res.status(400).json({ erros: msgs });
      }
      res.status(500).json({ erro: 'Erro ao criar produto' });
    }
  },

  async atualizar(req, res) {
    try {
      const [atualizado] = await Produto.update(req.body, {
        where: { id: req.params.id },
      });
      if (!atualizado) return res.status(404).json({ erro: 'Não encontrado' });
      const produto = await Produto.findByPk(req.params.id);
      res.json(produto);
    } catch (erro) {
      res.status(500).json({ erro: 'Erro ao atualizar produto' });
    }
  },

  async remover(req, res) {
    try {
      const deletado = await Produto.destroy({ where: { id: req.params.id } });
      if (!deletado) return res.status(404).json({ erro: 'Não encontrado' });
    }
  }
};
```

```
        res.status(204).send();
    } catch (erro) {
        res.status(500).json({ erro: 'Erro ao remover produto' });
    }
},
};

module.exports = ProdutoController;
```

10.3 Estrutura final do projeto

Estrutura Final

```
minha-api/
├── .sequelizerc
├── index.js
├── package.json
├── src/
│   ├── controllers/
│   │   └── ProdutoController.js
│   ├── database/
│   │   ├── config.json
│   │   ├── connection.js
│   │   └── migrations/
│   │       └── 20250101000000-criar-tabela-produtos.js
│   ├── middlewares/
│   │   ├── logger.js
│   │   └── validarProduto.js
│   ├── models/
│   │   └── Produto.js
│   └── routes/
│       └── produtos.js
```

11. Atividade Prática

Aplique os conceitos estudados neste material construindo uma API REST completa para o sistema descrito abaixo.

Contexto: API de Biblioteca

Construa uma API REST para gerenciar o acervo de uma biblioteca escolar.

A API deverá permitir o cadastro e consulta de livros do acervo.

11.1 Requisitos

O recurso Livro deve conter os seguintes atributos:

- id — inteiro, auto-incremento, chave primária
- titulo — string (até 200 caracteres), obrigatório
- autor — string (até 100 caracteres), obrigatório
- isbn — string (13 dígitos), único, obrigatório
- ano_publicacao — inteiro, entre 1800 e o ano atual
- quantidade — inteiro, maior ou igual a zero (padrão: 1)
- createdAt / updatedAt — automático pelo Sequelize

Implemente os seguintes endpoints:

Método	URL	Descrição
GET	/livros	Listar todos os livros
GET	/livros/:id	Buscar livro por ID
POST	/livros	Cadastrar novo livro
PUT	/livros/:id	Atualizar dados de um livro
DELETE	/livros/:id	Remover livro do acervo

11.2 O que deve ser implementado

1. Estrutura de pastas seguindo o padrão Model-Controller-Route
2. Migration criada e aplicada corretamente no PostgreSQL
3. Model Sequelize com validações nos campos titulo, autor, isbn e ano_publicacao
4. Todos os 5 endpoints implementados e funcionando corretamente
5. Middleware de log e middleware de validação implementados
6. Tratamento de erros: 404 para não encontrado, 400 para dados inválidos, 500 para erros internos